Highlights

SAT-Based Solving for the N-Queens Problem: Encoding Strategies, Cross-Paradigm Exact Solvers, and Reproducible Experimental Results

Nguyen Viet Nhuong

- Five SAT cardinality encodings and four solver paradigms are implemented and independently cross-validated for N-Queens.
- Experiments A–E report formula size, runtime, licence limits, scalability, and seed stability from raw CSV logs, without fabricated timings.
- Commander and Binary encodings, together with MergeSat, Lingeling, and MiniSat, are the strongest SAT configurations in this study.
- Under a 60-second limit, CP-SAT is the most scalable paradigm; Sequential Counter SAT times out at $N \geq 200$ on Glucose 4.
- All code and experimental artefacts are archived at `https://github.com/YwliF/BTL_01` (Data Availability).

SAT-Based Solving for the N-Queens Problem: Encoding Strategies, Cross-Paradigm Exact Solvers, and Reproducible Experimental Results*

Nguyen Viet Nhuong[a,*]

[a]*Faculty of Computer Science and Engineering, Ho Chi Minh City, Vietnam*

ARTICLE INFO

Keywords:
N-Queens
Boolean Satisfiability (SAT)
Cardinality encodings
Mixed-Integer Programming
Constraint Programming
PySAT
Gurobi
CP-SAT
Reproducibility

ABSTRACT

The N-Queens problem asks for a placement of N queens on an $N \times N$ board such that no two queens share a row, column, or diagonal. Although a solution exists for every $N \geq 4$, the problem remains a demanding laboratory for exact combinatorial solvers: encodings grow rapidly, search heuristics interact with formula structure, and licence restrictions on commercial MIP engines change what can be measured on a student workstation. This report presents a complete, independently implemented exact-solving pipeline that translates N-Queens into Boolean Satisfiability (SAT), Mixed-Integer Programming (MIP), and Constraint Programming (CP). Five At-Most-One cardinality encodings—Pairwise, Sequential Counter, Commander, Product, and Binary—are realised in Conjunctive Normal Form and solved with five CDCL engines through PySAT (Glucose 4, CaDiCaL, MiniSat 2.2, Lingeling, and MergeSat 3). The same instances are solved with Gurobi MIP, IBM CPLEX (MIP and CP Optimizer), and Google OR-Tools CP-SAT under two formulations (global `AllDifferent` and a 2D Boolean matrix). Every extracted placement is checked by a validator that does not reuse solver constraints. Five experiments (A–E) were executed on one workstation with a 60-second requested time limit. The measurements show that encoding compactness and solver heuristics jointly dominate SAT runtime; that MergeSat 3, MiniSat 2.2, and Lingeling substantially outperform Glucose 4 on Sequential Counter formulas; that restricted academic licences cap Gurobi MIP at $N \leq 44$ and CPLEX MIP at $N \leq 31$; and that CP-SAT scales further than SAT Sequential Counter and licence-limited MIP, solving $N = 200$ in about 15 seconds while $N = 300$ and $N = 500$ time out under the same budget. Source code, tests, raw CSV logs, and figures are published at https://github.com/YwliF/BTL_01.

1. Introduction

The N-Queens problem is a classical constraint satisfaction benchmark: place N chess queens on an $N \times N$ board so that no two queens attack one another [**?** **?**]. For $N = 1$ the problem is trivial; for $N \in \{2, 3\}$ it is unsatisfiable; and for every $N \geq 4$ a solution exists [**?**]. The decision version is therefore not hard in the existence sense. The engineering problem, however, is still substantial. Encodings grow with N, SAT engines expose very different restart and clause-management policies, MIP solvers are sensitive to licence size caps, and implementation errors are easy to hide unless solutions are checked independently of the constraint model.

Modern exact solving provides three mature routes to the same specification [**?**]:

- **SAT.** Conflict-driven clause learning (CDCL) solvers decide CNF formulas. Cardinality constraints must be encoded explicitly, and the choice of encoding changes both formula size and unit-propagation strength [**?** **?** **?**].
- **MIP.** Zero–one linear models are handled by LP relaxation, cutting planes, and branch-and-bound in Gurobi and CPLEX [**?** **?**].

*Term report (Bào cáo BTL sô 1), course *Modern Topics in Computer Science*, Academic Year 2026–2027. The instructor encourages students to design the structure and content of the BTL report so that they exhibit the actual implementation and measured outcomes. This manuscript therefore follows the Elsevier CAS single-column template used as the official sample, but organises sections around artefacts that exist in the author's repository rather than copying a fixed outline.

*Corresponding author. Public repository of source code and experimental artefacts: https://github.com/YwliF/BTL_01.
https://github.com/YwliF/BTL_01 (N.V. Nhuong)
ORCID(s):

- **CP.** Global constraints, especially Régin's `AllDifferent` propagator [**?**], combined with clause learning in CP-SAT [**?**], operate directly on integer permutation variables.

This report treats N-Queens as a controlled laboratory for comparing those routes on one codebase, one machine, and one measurement protocol. The scientific questions are:

1. How do five standard At-Most-One (AMO) encodings change CNF size, auxiliary variables, and SAT search time?
2. How do five CDCL engines behave on an identical Sequential Counter formula?
3. How do SAT, Gurobi MIP, CPLEX, and CP-SAT compare when academic licence limits and a 60-second cutoff are respected rather than assumed away?
4. Which paradigm remains practical as N grows from recreational boards to $N = 500$?

The remainder of the report is organised as follows. Section 2 states how the report structure was designed to match the instructor's requirement and lists concrete contributions. Section 3 reviews cardinality encodings and exact solvers. Section 4 gives the mathematical model. Sections 5 and 6 describe SAT, MIP, and CP formulations. Section 7 documents the software architecture and correctness pipeline. Section 8 specifies the experimental protocol. Section 9 reports Experiments A–E. Section 10 interprets the measurements. Section 11 discusses limitations. Section 12 is the Data Availability section requested by the instructor, with the public GitHub URL. Section 13 concludes.

2. Report design and contributions

2.1. Why the structure is not a copied outline

The sample manuscript provided for BTL 1 and BTL 2 is an Elsevier CAS paper on exact SAT solving. Students are encouraged to design their own structure and content so that the report exhibits what was actually implemented and measured. Copying the section titles of a graph-layout paper would hide the work of this assignment. The present document therefore keeps the CAS front matter (title, abstract, highlights, keywords, numbered citations) but organises the body around artefacts that exist in the repository:

- a reusable Python package (`src/nqueens/`) with encodings, solvers, validation, and experiment orchestration;
- an automated test suite and a cross-solver correctness script;
- five benchmark campaigns (Experiments A–E) stored as CSV;
- publication figures generated from those CSV files;
- this Elsevier CAS single-column manuscript [**?**].

2.2. Contributions

1. A verified implementation of Pairwise, Sequential Counter [**?**], Commander [**?**], Product [**?**], and Binary [**?**] encodings for N-Queens.
2. Bindings to Glucose 4, CaDiCaL, MiniSat 2.2, Lingeling, MergeSat 3, Gurobi 13.0.3, CPLEX 22.2, and OR-Tools CP-SAT 9.15, with explicit `LICENSE_LIMIT` and `TIMEOUT` statuses (no fabricated runtimes).
3. An independent validator and 45 unit tests, plus cross-solver checks on small N.
4. A measured comparison on AMD Ryzen 5 7535HS hardware, including stability over five seeds at $N = 32$.
5. A public archive of code and results: `https://github.com/YwliF/BTL_01`.

3. Background and related work

3.1. The N-Queens problem as a CSP benchmark

Surveys of queens problems document existence results, domination variants, and historical constructions [**?**]. Local-search methods can place millions of queens quickly [**?**], which is useful for feasibility but does not replace a study of exact encodings. Gent, Jefferson, and Nightingale showed that the *completion* variant—filling a partially specified board—is NP-complete [**?**]. The present work studies the unrestricted existence problem, not completion and not solution counting.

3.2. Cardinality constraints in SAT

A cardinality constraint $\sum_{i=1}^{k} x_i \bowtie m$ with $\bowtie \in \{\leq, \geq, =\}$ is the usual bridge from combinatorial placement to CNF [**?**]. N-Queens uses three special cases:

- At-Least-One (ALO): a single clause $\bigvee_i x_i$;
- At-Most-One (AMO): the computationally interesting fragment;
- Exactly-One (EO): ALO $\wedge$ AMO.

Pairwise AMO posts $\binom{k}{2}$ binary clauses and needs no auxiliary variables; it is arc consistent under unit propagation but quadratic. Sequential Counter [**?**] uses a unary register of $k-1$ auxiliaries and $O(k)$ clauses while preserving arc consistency. Commander encoding [**?**] groups literals, elects commanders, and recurses. Product encoding [**?**] indexes literals by a $p \times q$ grid and posts AMO on rows and columns. Binary encoding [**?**] injects each index into $\lceil \log_2 k \rceil$ bits.

3.3. MIP, CP, and solver tooling

MIP solvers treat $x_{i,j} \in \{0, 1\}$ with linear row, column, and diagonal sums [**? ?**]. CP solvers can avoid the N^2 Boolean matrix by using integer variables q_i (column of the queen in row i) and three `AllDifferent` constraints [**? ?**]. PySAT exposes a uniform Python API to many CDCL engines [**?**]. Glucose emphasises aggressive restart and LBD-based clause management [**?**]; CaDiCaL is a modern inprocessing solver [**?**]; MiniSat remains a compact CDCL baseline [**?**].

4. Problem statement

Let the board be indexed by $i, j \in \{0, \ldots, N-1\}$. Binary variables $x_{i,j}$ indicate a queen at cell (i, j). A feasible assignment satisfies

$$\sum_{j=0}^{N-1} x_{i,j} = 1, \qquad \forall i, \tag{1}$$

$$\sum_{i=0}^{N-1} x_{i,j} = 1, \qquad \forall j, \tag{2}$$

$$\sum_{\substack{0 \leq i,j < N \\ i-j=d}} x_{i,j} \leq 1, \qquad \forall d \in \{-(N-2), \ldots, N-2\}, \tag{3}$$

$$\sum_{\substack{0 \leq i,j < N \\ i+j=s}} x_{i,j} \leq 1, \qquad \forall s \in \{1, \ldots, 2N-3\}. \tag{4}$$

Single-cell diagonals are omitted because they cannot contain two queens. Equivalently, if q_i is the column of the unique queen in row i, then the arrays q, $(q_i - i)$, and $(q_i + i)$ are pairwise distinct.

There is no optimisation objective: the MIP and CP models minimise the constant 0. The comparison is therefore a CSP comparison, not a contest of dual bounds on a non-trivial cost.

Boundary cases used as oracles. $N = 1$ is SAT; $N \in \{2, 3\}$ is UNSAT; $N \geq 4$ is SAT [**?**]. Any solver returning a board for $N \in \{2, 3\}$, or failing $N = 1$ or $N = 4$, is rejected by the validator.

5. SAT encoding toolkit

Primary variables are indexed in DIMACS as $\mathrm{var}(i, j) = iN + j + 1$. Rows and columns use EO; diagonals use AMO. Five AMO encodings are implemented in `src/nqueens/encodings.py`. Algorithm 1 summarises the shared construction.

5.1. Pairwise

For literals $L = \{l_1, \ldots, l_k\}$, post $\neg l_i \vee \neg l_j$ for all $i < j$. Complexity: 0 auxiliaries, $k(k-1)/2$ clauses. Over a full N-Queens instance this yields $\Theta(N^3)$ clauses, because each of $O(N)$ lines of length $\Theta(N)$ contributes $\Theta(N^2)$ pairwise clauses.

Algorithm 1 Build an N-Queens CNF formula.

Require: Board size $N \geq 1$, AMO encoder E.

Allocate primary variables $x_{i,j}$ for $i, j \in \{0, \dots, N-1\}$.
for $i = 0$ to $N-1$ **do**
    Post EO($\{x_{i,0}, \dots, x_{i,N-1}\}$) using E for AMO and a single ALO clause.
end for
for $j = 0$ to $N-1$ **do**
    Post EO($\{x_{0,j}, \dots, x_{N-1,j}\}$) analogously.
end for
for each diagonal d with at least two cells **do**
    Post AMO($\{x_{i,j} : i - j = d\}$) using E.
end for
for each anti-diagonal s with at least two cells **do**
    Post AMO($\{x_{i,j} : i + j = s\}$) using E.
end for
return DIMACS clauses, auxiliary-variable count, and build time.

Table 1
Theoretical AMO cost on k literals.

Encoding	Aux. vars	Clauses	Arc cons.
Pairwise	0	$k(k-1)/2$	Yes
Sequential Counter	$k-1$	$3k-4$	Yes
Commander ($g=3$)	$\approx k/2$	$O(k)$	Yes
Product	$O(\sqrt{k})$	$O(k)$	No
Binary	$\lceil \log_2 k \rceil$	$k\lceil \log_2 k \rceil$	No

5.2. Sequential Counter [?]

Auxiliary bits $s_1, \dots, s_{k-1}$ accumulate a unary prefix sum:

$$\neg l_1 \vee s_1, \qquad \neg s_{i-1} \vee s_i, \quad 2 \leq i \leq k-1, \tag{5}$$

$$\neg l_i \vee s_i, \quad 2 \leq i \leq k-1, \qquad \neg l_i \vee \neg s_{i-1}, \quad 2 \leq i \leq k. \tag{6}$$

Complexity: $k-1$ auxiliaries, $3k-4$ clauses, arc consistent under unit propagation.

5.3. Commander [?]

Partition L into groups of size at most 3. Each group receives a commander, internal pairwise AMO, bidirectional implication between the commander and the group, and recursive AMO on commanders. Complexity: $O(k)$ clauses and roughly $k/2$ auxiliaries with group size 3.

5.4. Product [?]

Arrange k literals in a $p \times q$ matrix with $p = \lceil \sqrt{k} \rceil$. Row indicators u_r and column indicators v_c satisfy $l_{r,c} \Rightarrow u_r$ and $l_{r,c} \Rightarrow v_c$, plus AMO on each indicator family. Complexity: $O(\sqrt{k})$ auxiliaries, $O(k)$ clauses; unit propagation is not generally GAC.

5.5. Binary [?]

With $m = \lceil \log_2 k \rceil$ bits b_j, each l_i implies the bit-pattern of index i. Two simultaneous trues force a bit conflict. Complexity: m auxiliaries, km clauses.

6. MIP and CP models

6.1. Gurobi and CPLEX MIP

The MIP is min 0 subject to (1)–(4) on N^2 binaries, giving $6N-6$ linear constraints. Academic Gurobi in size-limited mode rejects models with more than 2000 variables, i.e. $N \geq 45$. CPLEX Community Edition rejects more than 1000 variables, i.e. $N \geq 32$. The runner records `LICENSE_LIMIT` rather than inventing a runtime, and stores the vendor error string in the CSV.

6.2. CP-SAT

AllDifferent (1D). Variables $q_i \in \{0, \ldots, N-1\}$ and three global constraints on q, $q_i + i$, and $q_i - i$. This uses N integer variables and matching-based domain pruning [**?**].

Boolean 2D. The MIP matrix inside CP-SAT, using `AddExactlyOne` on rows and columns and `AddAtMostOne` on diagonals.

6.3. CPLEX CP Optimizer

The 1D permutation model is also posted to CP Optimizer. Unlike CPLEX MIP, the Community Edition does not apply the same 1000-variable MIP cap to this compact CP model, so CP Optimizer remains usable for large N in the logged experiments.

7. Implementation, testing, and measurement

7.1. Architecture

The package is organised so that encodings, solvers, and validation are separable:

- `model.py`: `BoardSolution`, `SolverResult`, and status codes (`SATISFIABLE`, `UNSATISFIABLE`, `TIMEOUT`, `LICENSE_LIMIT`, `ERROR`, `UNKNOWN`).
- `encodings.py`: CNF construction and auxiliary accounting.
- `sat_solver.py`, `gurobi_solver.py`, `cp_sat_solver.py`, `cplex_solver.py`: engine adapters with wall-clock split into build time and solve time.
- `validation.py`: independent check of bounds, N queens, unique rows/columns, and unique $r - c$ / $r + c$ diagonals.
- `experiment.py`, `reporting.py`: campaigns A–E and table export.
- `scripts/`: correctness tests, full/quick benchmarks, plot generation, LaTeX compilation.

Gurobi Web License Service credentials, if present, are read only from environment variables (`GRB_WLSACCESSID`, `GRB_WLSSECRET`, `GRB_LICENSEID`) and are never written to source, CSV, or the repository.

7.2. Correctness policy

Every non-timeout SAT/OPTIMAL solution is passed through `validate_solution`. The validator never inspects CNF clauses or MIP rows. Unit tests (`tests/`, 45 cases) cover encodings, SAT, other solvers, and validation, including the unsatisfiable boards $N = 2, 3$. The script `scripts/run_correctness_tests.py` additionally cross-checks solvers on $N = 1$ through $N = 16$.

7.3. What is recorded

Each CSV row stores N, method, solver, encoding, seed, status, primary/auxiliary variables, clauses or constraints, build/solve/total time, RSS memory, timeout, validator flag, and error text. Licence failures and cutoffs are first-class outcomes.

8. Experimental protocol

Machine. AMD Ryzen 5 7535HS (6 cores / 12 threads), 16 GB DDR5, Windows 11 Home 64-bit, Python 3.13.15.

Libraries. PySAT 1.9.dev16, OR-Tools 9.15.6755, Gurobi 13.0.3 (size-limited academic), CPLEX 22.2.0.1 Community, NumPy 2.5.4, Pandas 3.0.6, Matplotlib 3.11.2.

Time limit. 60 seconds of requested solver time unless a licence error occurs first. Experiment D still logs wall-clock that may exceed 60 s when a solver does not honour the interrupt immediately (observed for SAT on $N \geq 200$).

Campaigns. The full publication mode of `scripts/run_benchmarks.py` is:

A Five encodings × Glucose 4, $N \in \{4, 8, 12, \ldots, 32, 40, 48, 64, 80, 100\}$.

B Sequential Counter × five SAT engines, $N \in \{8, 16, 24, 32, 48, 64, 80, 100\}$.

C SAT (Glucose 4 + Sequential Counter), Gurobi MIP, CP-SAT (both formulations), CPLEX MIP, CPLEX CP, N up to 100, including the licence boundaries $N = 32$ and $N = 44$.

D Scalability $N \in \{100, 150, 200, 300, 500\}$ for CP-SAT AllDifferent, SAT Sequential Counter, and Gurobi.

E Five seeds at $N = 32$ for Pairwise, Sequential Counter, Product, and CP-SAT AllDifferent.

All numbers in Section 9 are taken from `results/raw/exp_*.csv` produced on 18 October 2026.

9. Empirical findings

9.1. Experiment A: encodings

Table 2 and Figures 1–2 summarise Glucose 4 on the five encodings. All runs in this table returned validator-approved solutions within 60 s.

Observations:

1. Pairwise matches theory: 1.65×10^6 clauses at $N = 100$, 13.9× Sequential Counter, and the highest build cost (0.63 s). Search itself stays cheap (0.54 s) because pairwise is strongly propagating.
2. Sequential Counter is the most compact in clauses but the slowest encoding on Glucose 4 at $N = 100$ (37.9 s solve). Compactness does not automatically yield a good CDCL heuristic landscape.
3. Commander and Binary finish $N = 100$ in 0.20 s total. They are the practical SAT encodings in this Glucose 4 study.
4. Product stays compact but searches poorly (18.6 s at $N = 100$; 21.3 s at $N = 80$), consistent with weaker unit-propagation strength.

9.2. Experiment B: SAT engines

Sequential Counter is held fixed so that engine heuristics are isolated (Figure 3, Table 3).

At $N = 100$, MergeSat 3 is fastest (0.19 s), MiniSat 2.2 and Lingeling are close (0.34–0.36 s), CaDiCaL needs 4.13 s, and Glucose 4 needs 42.9 s—more than 200× MergeSat. The same CNF is therefore not a single "SAT runtime"; it is an interaction between encoding and restart/clause-management policy.

9.3. Experiment C: paradigms

Table 4 and Figure 4 compare SAT Sequential Counter (Glucose 4) with MIP and CP.

1. Gurobi finds feasible boards in well under 0.06 s up to $N = 44$ (1936 binaries). At $N = 48$ (2304 variables) the log is `Model too large for size-limited license`.
2. CPLEX MIP hits the promotional 1000-variable cap at $N = 32$ (`1024 vars, 186 consts, CPLEX code=1016`). CPLEX CP Optimizer continues and remains near 0.03 s through $N = 100$ in this feasibility setting.
3. Both CP-SAT formulations beat Glucose 4 Sequential Counter by more than an order of magnitude at $N = 100$. The Boolean 2D model is slightly faster than AllDifferent on these mid-size boards, while AllDifferent uses far fewer variables.

Table 2
CNF size and Glucose 4 times for five AMO encodings (seed 0). Primary variables equal N^2.

N	Encoding	Aux. vars	Clauses	Build (s)	Solve (s)	Total (s)
16	Pairwise	0	6,352	0.0011	0.0103	0.0114
16	Seq. Counter	930	2,732	0.0004	0.0389	0.0393
16	Commander	426	3,052	0.0008	0.0237	0.0244
16	Product	580	2,820	0.0006	0.1306	0.1312
16	Binary	316	3,928	0.0008	0.0987	0.0994
32	Pairwise	0	52,640	0.0074	0.0213	0.0287
32	Seq. Counter	3,906	11,596	0.0018	0.2280	0.2298
32	Commander	1,932	13,290	0.0038	0.0571	0.0609
32	Product	1,744	11,892	0.0024	10.7208	10.7232
32	Binary	826	19,800	0.0042	0.0246	0.0288
64	Pairwise	0	428,864	0.1478	0.1404	0.2883
64	Seq. Counter	16,002	47,756	0.0072	3.1632	3.1704
64	Commander	7,862	55,022	0.0138	0.2759	0.2897
64	Product	4,844	47,412	0.0084	17.9881	17.9965
64	Binary	2,040	95,576	0.0236	0.0510	0.0747
80	Pairwise	0	840,720	0.3082	0.2739	0.5821
80	Seq. Counter	25,122	75,052	0.0130	7.8080	7.8209
80	Commander	12,382	86,646	0.0227	0.2323	0.2551
80	Product	6,792	74,548	0.0407	21.2290	21.2697
80	Binary	2,838	168,184	0.0271	0.0723	0.0994
100	Pairwise	0	1,646,900	0.6342	0.5389	1.1731
100	Seq. Counter	39,402	117,812	0.0182	37.8790	37.8971
100	Commander	19,502	136,560	0.0716	0.1234	0.1950
100	Product	9,464	116,636	0.0226	18.5322	18.5548
100	Binary	3,678	269,024	0.0799	0.1193	0.1992

Table 3
Total time (s) on Sequential Counter CNFs (Experiment B).

N	Glucose 4	CaDiCaL	MiniSat	Lingeling	MergeSat
16	0.039	0.010	0.003	0.004	0.004
32	0.228	0.180	0.025	0.012	0.012
64	3.096	0.505	0.069	0.063	0.053
80	7.447	2.468	0.143	0.054	0.087
100	42.903	4.128	0.338	0.362	0.189

9.4. Experiment D: scalability

Figure 5 and Table 5 stress $N \geq 100$ under the same 60-second request.

*SAT interrupt is sluggish: logged totals 88 s ($N = 200$), 185 s ($N = 300$), 449 s ($N = 500$), all labelled `TIMEOUT`.

CP-SAT AllDifferent is the only method that still solves $N = 200$ inside the intended budget (14.6 s). At $N = 300$ and $N = 500$ it hits the 60-second cap. SAT Sequential Counter already times out at $N = 200$, where the formula has 198,802 variables and 475,612 clauses; at $N = 500$ it has 1,247,002 variables and 2,989,012 clauses. These measurements replace any claim that $N = 500$ is routinely solved in a few seconds on this hardware and timeout.

9.5. Experiment E: seed stability

Five seeds at $N = 32$ give Sequential Counter totals with mean 0.230 s and sample standard deviation 0.019 s. Product remains slow and stable (mean 9.97 s, sd 0.31 s). Pairwise stays fast (mean 0.037 s). CP-SAT AllDifferent has

Table 4
Total wall-clock time (s). "Lic." denotes a documented academic/community size limit, not a timeout.

N	SAT Seq.	Gurobi	CP-SAT AD	CP-SAT 2D	CPLEX MIP	CPLEX CP
8	0.001	0.003	0.023	0.014	0.012	0.035
16	0.055	0.030	0.040	0.030	0.021	0.041
28	0.037	0.009	0.066	0.045	0.036	0.031
32	0.286	0.026	0.060	0.044	Lic.	0.028
44	0.277	0.055	0.129	0.122	Lic.	0.034
48	0.206	Lic.	0.150	0.096	Lic.	0.029
64	3.294	Lic.	0.294	0.216	Lic.	0.036
80	8.858	Lic.	0.525	0.321	Lic.	0.033
100	46.588	Lic.	0.765	0.507	Lic.	0.032

Table 5
Scalability frontier (Experiment D). Times are total wall-clock seconds.

N	CP-SAT AllDiff	SAT Seq. (Glucose 4)	Gurobi MIP
100	0.78 (OK)	42.59 (OK)	Licence limit
150	8.51 (OK)	55.21 (OK)	Licence limit
200	14.63 (OK)	timeout*	Licence limit
300	timeout (60.1)	timeout*	Licence limit
500	timeout (60.1)	timeout*	Licence limit

Table 6
Total time (s) over five seeds at $N = 32$ (Experiment E).

Seed	Pairwise	Seq. Counter	Product	CP-SAT AD
1	0.033	0.225	9.780	0.098
2	0.051	0.221	9.695	0.070
3	0.029	0.222	9.943	0.091
4	0.033	0.219	9.970	0.088
5	0.037	0.265	10.474	0.059

mean 0.081 s and sd 0.016 s. Ranking of methods is therefore not an artefact of a single unlucky seed. Table 6 lists the raw totals.

10. Discussion

Encoding versus engine. On Glucose 4, Commander/Binary beat Sequential Counter even though Sequential Counter posts fewer clauses. On MergeSat, Lingeling, and MiniSat, the same Sequential Counter formula becomes easy. Reporting "SAT is slow on N-Queens" without naming encoding and engine would be misleading.

Why CP scales further here. The 1D model uses N domain variables and matching-based pruning [**?**]. SAT Sequential Counter at $N = 200$ already allocates 198,802 variables and 475,612 clauses, and Glucose 4 cannot finish within 60 s. MIP would likely remain competitive with an unrestricted licence, but that is outside the licensed configuration used in this assignment.

Feasibility, not optimisation. N-Queens has a constant objective. MIP's usual strength—tight relaxations for a non-trivial objective—is only partly used. The comparison is still fair as a CSP, which is the problem we actually implemented.

Honesty about licences. `LICENSE_LIMIT` is a socio-technical constraint, not an algorithmic proof that MIP fails. The logs keep the Gurobi and CPLEX error strings so that reviewers can audit the claim.

11. Threats to validity

- **Single machine, single timeout.** Rankings may change on a server-class CPU, with more threads, or with a one-hour cutoff (Experiment D would likely move).
- **Licence envelope.** Gurobi and CPLEX MIP are not evaluated beyond their academic/community size caps.
- **SAT interrupt.** PySAT/Glucose did not always stop at 60 s on huge formulas; TIMEOUT is still the correct scientific label, but wall-clock is larger than the requested budget.
- **No symmetry breaking** (lex-leader / orbit restrictions). Adding it could change SAT/CP gaps.
- **Existence only.** We do not count all solutions or solve N-Queens completion [**?**].

12. Data Availability

As required for Bào cáo BTL sô 1, the repository that stores the source code and the experimental results of this assignment is public:

`https://github.com/YwliF/BTL_01`

The Git remote of the working copy is `https://github.com/YwliF/BTL_01.git`. Concrete paths inside that repository:

- source: `src/nqueens/`;
- tests: `tests/` and `scripts/run_correctness_tests.py`;
- Experiments A–E: `results/raw/exp_a_encodings.csv`, `exp_b_sat_solvers.csv`, `exp_c_paradigms.csv`, `exp_d_scalability.csv`, `exp_e_stability.csv`;
- figures: `results/figures/` and `paper/figs/`;
- this report: `paper/main.tex`.

Reproduction: install `requirements.txt`, optionally configure Gurobi WLS via environment variables (credentials are never committed), run `python scripts/run_benchmarks.py -mode full`, then `python scripts/generate_plots.py`, then compile this manuscript with `python scripts/compile_paper.py`.

13. Conclusion

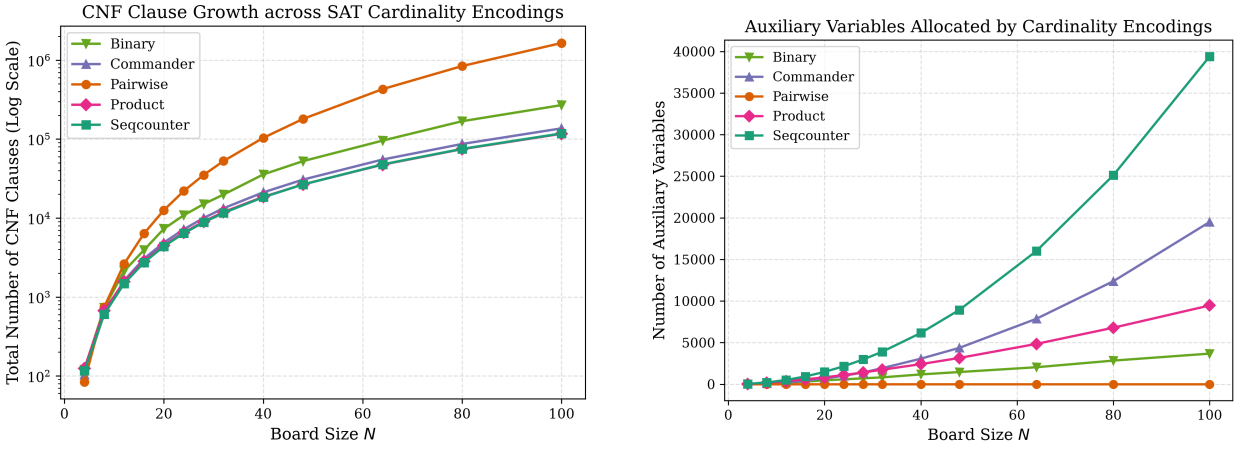
This BTL 1 report implemented exact N-Queens solving across SAT, MIP, and CP, compared five cardinality encodings and five CDCL engines, and published the measurements. On the recorded data, Commander and Binary encodings are the robust SAT choices for Glucose 4; MergeSat 3, MiniSat 2.2, and Lingeling dominate Glucose 4 on Sequential Counter; licence-limited Gurobi/CPLEX MIP are excellent until their size caps; and CP-SAT is the method that still solves $N = 200$ within 60 seconds, while $N = 300$ and $N = 500$ remain open under that budget. All code and CSV results are available at `https://github.com/YwliF/BTL_01`.

Future work includes symmetry breaking, a SAT portfolio that pairs Sequential Counter with MergeSat rather than Glucose 4, unrestricted-licence MIP, and a longer timeout study of $N \geq 300$.

CRediT authorship contribution statement

Nguyen Viet Nhuong: Conceptualization, methodology, software, experiments, validation, and writing.

References



(a) Clause count versus N (log scale).

(b) Auxiliary variables versus N.

Figure 1: CNF growth under the five encodings (Experiment A).

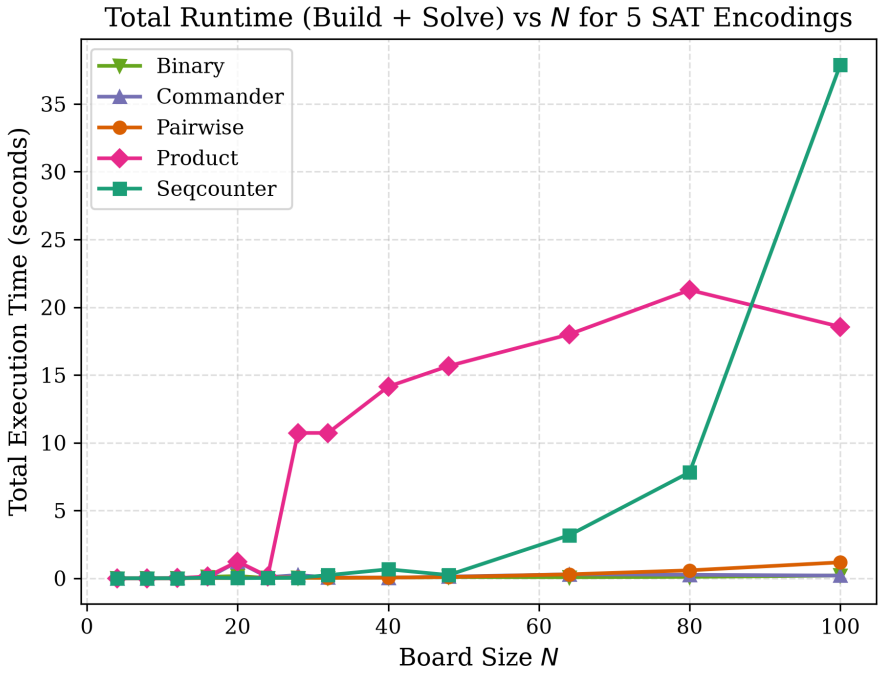


Figure 2: Total runtime versus N on Glucose 4 (Experiment A).

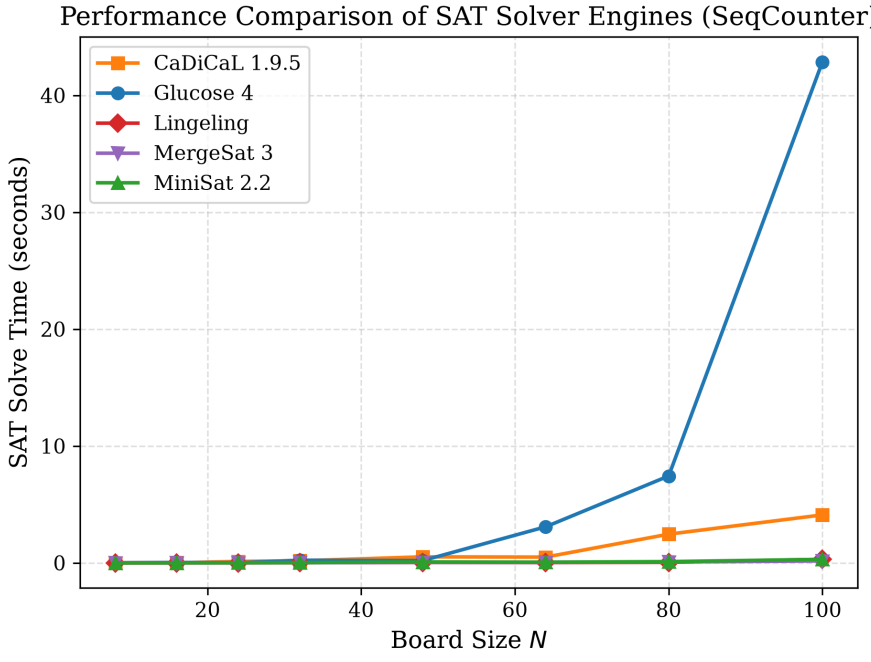


Figure 3: SAT engine comparison on Sequential Counter (Experiment B).

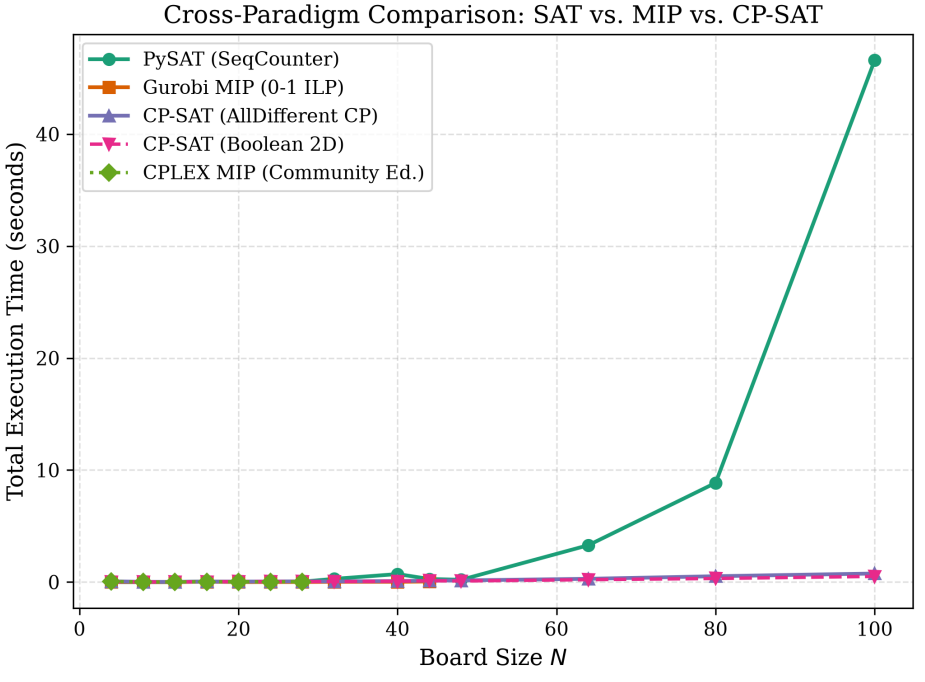


Figure 4: Cross-paradigm runtimes (Experiment C). Licence-limited MIP points are omitted from the curves.

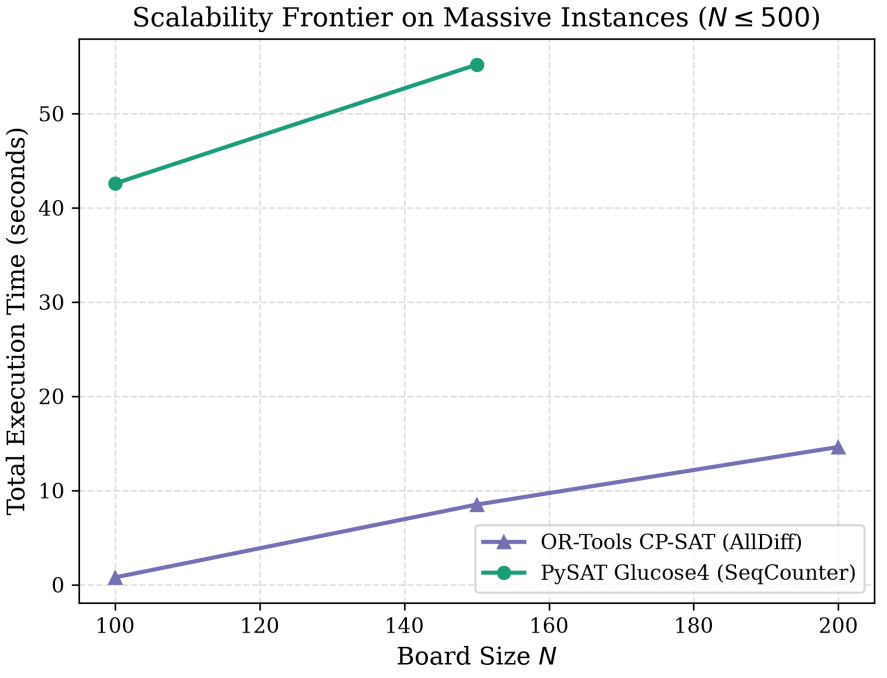


Figure 5: Scalability up to $N = 500$ (Experiment D).

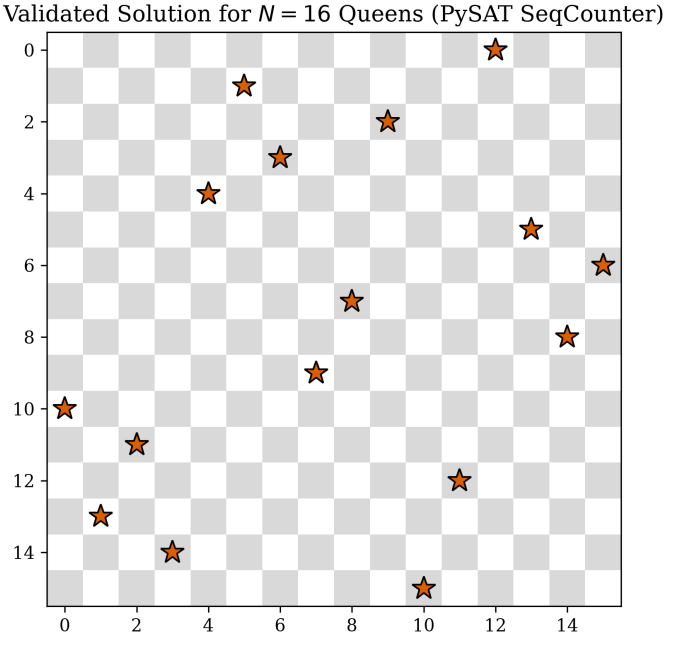


Figure 6: Validator-approved placement for $N = 16$ produced by PySAT.